

SIH26140 — Quantum Intelligence Learning Lab

Prototype Master Specification — Concise Summary

1. Product Overview

Working expansion: Eureka Forge — Quantum Intelligence Learning Lab. The product is an interactive learning platform for undergraduate/early-postgraduate learners studying introductory gate-model quantum computing.

Promise: "Students predict. The simulator proves. AI explains why."

Core learning loop: Predict → Build → Simulate → Compare → Diagnose → Explain → Show Me Why → Challenge → Mastery.

MVP curriculum: Qubit, Superposition, H gate, Measurement, CNOT, Entanglement, Bell state.

2. Core Architecture

Non-negotiable principle: "Simulator computes. Everything else reads."

Learner Action → Canonical Circuit Representation → Qiskit Aer → Structured Execution Trace → Visualizer / Assessment / AI Tutor / Manim Selector.

- Qiskit Aer is the source of quantum-result truth.
- The LLM must not independently calculate or invent circuit results.
- AI receives structured circuit, prediction, simulation, trace, and assessment context.
- The exact schema may evolve, but the source-of-truth contract must remain.

3. MVP Technology Stack

- Frontend: React / Next.js, TypeScript, Tailwind CSS, SVG/Canvas; Three.js only where useful.
- Backend: Python, FastAPI, Qiskit Aer.
- AI: Ollama local-first + Gemini fallback behind one provider interface.
- Animation: Manim Community Edition using pre-rendered MVP clips.
- Data: SQLite for MVP; PostgreSQL later.
- Future adapters: PennyLane, Cirq, qBraid/other environments, and real QPU execution.

4. Required Learner Journey

1. Dashboard — learning path, progress, concept cards, mastery indicators.
2. Lesson — concept → visual intuition → mini explanation → predict → experiment → challenge.
3. Prediction — learner submits a prediction before simulation.
4. Circuit Lab — H, X, Z, CNOT/CX, Measurement; add/remove/clear/run/step through.
5. Simulation Result — probability histogram, shots, circuit summary, state/probability view.
6. Compare — prediction versus simulator result with deterministic match/mismatch.
7. Step-by-Step Trace — execution steps generated from the execution pipeline.
8. AI Tutor — Explain, Hint, Debug using supplied evidence.
9. Misconception Engine — deterministic MVP rules.

10. Show Me Why — pre-rendered Manim explanations.
11. Challenge — three graded tasks.
12. Mastery — Learning / Practicing / Mastered states based on actual performance.

5. Misconception Engine

- M1 — Superposition as a hidden classical value.
- M2 — Measurement does not change state.
- M3 — Entanglement as faster-than-light communication.
- M4 — Phase equals probability.

These are MVP teaching heuristics, not an exhaustive scientific taxonomy. The rules are deterministic and should expose trigger, evidence, strength/confidence, learner explanation, remediation concept, suggested challenge, and optional Manim clip.

6. Grounded AI Tutor

- Receives learner question, canonical circuit, simulator result, prediction, comparison, execution trace, misconception result, and curated lesson context.
- Never invents simulator results.
- Never overrides deterministic grading.
- States when evidence is unavailable.
- Explains using supplied execution evidence.
- Hint mode should not simply reveal the answer.
- Debug mode identifies the supplied mismatch.
- Simulator facts and conceptual explanation remain distinct.
- Use Ollama locally when configured and Gemini as fallback; mock the LLM in tests.

7. Show Me Why — Manim

- H gate → superposition.
- Measurement → observed outcome/state change.
- H + CNOT → Bell-state correlation.
- Use metadata for clip ID, concept, misconception IDs, title, and description.
- Frontend selects clips deterministically from concept/misconception context.
- No live Manim generation is required; provide a static fallback.

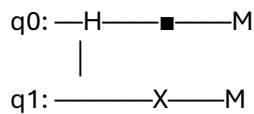
8. Key Test Circuits

$H|0\rangle$:

$|0\rangle \approx 50\% \quad |1\rangle \approx 50\%$

Use tolerance because shot-based simulation is stochastic.

Bell state:



$P(00) \approx 50\%$, $P(11) \approx 50\%$, $P(01) \approx 0\%$, $P(10) \approx 0\%$.

9. Implementation Order

13. Phase 1 — Foundation: audit, canonical circuit model, Aer, execution-result schema, golden tests.
14. Phase 2 — Core learning loop: prediction, simulation, comparison, step trace, visualization.
15. Phase 3 — Intelligence: misconception engine, grounded AI tutor, mastery engine.
16. Phase 4 — Experience: Manim clips, dashboard, lesson, challenge flow.
17. Phase 5 — SIH polish: UI polish, reliability audit, demo rehearsal.

10. SIH Demo Script

18. Dashboard — establish that quantum concepts are difficult to see.
19. Superposition lesson.
20. Student prediction.
21. H circuit simulation.
22. Prediction versus simulator ground truth.
23. AI explains the mismatch using circuit and execution evidence.
24. Show Me Why visual explanation.
25. Bell-state application.
26. Challenge selected from actual performance.
27. Mastery becomes measurable and iterative.

Closing line: "Students predict. The simulator proves. AI explains why."

11. Explicitly Out of Scope for MVP

- Shor, QFT, QAOA and a full curriculum.
- Real QPU integration.
- Four fully working backends.
- Instructor dashboard and classroom collaboration.
- Advanced gamification and social features.
- Dynamic AI-generated Manim.
- 20+ qubit simulation.
- Unnecessary 3D worlds.

12. Prototype Acceptance Criteria

- Lesson starts and prediction can be submitted and stored.
- $H|0\rangle$ runs through Aer and actual output is visualized.
- Prediction comparison is deterministic.
- Step-through works.
- A mismatch can trigger a misconception rule.

- AI receives structured simulator evidence and produces an explanation.
- Show Me Why opens the correct clip.
- Challenge can be completed and mastery changes from actual performance.
- Bell-state journey works.
- No critical route errors; tests pass; local-first demo works.

13. Evidence & Honesty Contract

Built = feature works in the submitted prototype.

Architected = data model/interface supports it, but it is not demonstrated.

Future = not implemented.

Never convert architecture into implementation, intention into result, simulation into hardware execution, AI explanation into verified physics reasoning, or planned research into completed research.

14. Future Roadmap & Research

- Phase 2: PennyLane/Cirq adapters, additional algorithms, noise-aware learning, code exercises, larger lesson library.
- Phase 3: real QPU execution, hardware-aware visualization, instructor dashboards, classroom analytics, collaboration, richer adaptive curriculum.
- Research questions: whether prediction-before-run improves understanding; whether misconception remediation improves prediction accuracy; whether simulator-grounded tutoring reduces unsupported explanations; and whether Show Me Why improves retention.

These outcomes should be evaluated rather than claimed before measurement.

15. Final SIH Verification

Before showing the PPT team anything, verify that the exact story on the slides can be demonstrated. If the PPT claims prediction, demonstrate prediction. If it claims simulator-grounded AI, demonstrate the relationship between simulator output and AI context. If it claims misconception detection, trigger one. If it claims Manim remediation, open one. If it claims mastery, show it changing.

If a claim cannot be demonstrated, downgrade it to Architected or Future scope.